

Deep Bluffing: A Heads-Up Limit Hold'em Poker Agent

ICG Summer Project 2026–27 — Capstone Report

Aryaa
Roll Number: 250210

July 19, 2026

Abstract

This report describes the design and implementation of an autonomous agent for Heads-Up Limit Texas Hold'em (HULHE), built entirely from scratch in Python without reliance on external poker libraries. The system consists of four layered components: (i) an exact combinatorial hand evaluator built on `itertools.combinations` and `collections.Counter`, (ii) a fast preflop hand-strength estimator based on the Chen Formula, (iii) a Monte Carlo rollout estimator for postflop win probability, and (iv) a Counterfactual Regret Minimization (CFR) strategy trained offline over a card-abstracted version of the game. Decisions are made primarily by the trained CFR strategy, with the Monte Carlo / expected-value heuristic serving as a provably safe fallback for any game state not covered during training. Local simulation against a random baseline opponent shows a positive win rate of roughly +0.8 big blinds per hand.

Contents

1	Introduction	2
2	Hand Evaluation	2
3	Preflop Strategy: The Chen Formula	3
4	Postflop Strategy: Monte Carlo Rollouts	3
5	Decision Rule: Pot Odds and Expected Value	4
5.1	Controlled Bluffing	4
6	Counterfactual Regret Minimization	4
6.1	Motivation and Abstraction	4
6.2	Algorithm: External-Sampling Monte Carlo CFR	5
6.3	Inference-Time Integration	6
7	Results	7
8	Limitations and Future Work	7
9	Conclusion	7

1 Introduction

Heads-Up Limit Hold'em is a well-studied benchmark in imperfect-information game theory: unlike perfect-information games such as Tic-Tac-Toe or the CartPole/FrozenLake environments used in earlier assignments of this course, a poker agent must act under genuine uncertainty about its opponent's private information. The goal of this project was to build both the game engine primitives (card representation, hand evaluation) and a competitive decision policy, without using pre-built poker libraries such as `treys` or `RLCard`.

The final agent is organized into independent, testable modules:

- `card.py` — a `Card` class with rank/suit parsing and comparison dunder methods.
- `deck.py` — a standard 52-card deck with dealing utilities.
- `hand_evaluator.py` — exact 5-card and best-of-7-card hand ranking.
- `agent.py` — the required `BasePokerBot` / `CustomPokerBot` entry point, containing the full decision logic.
- `cfr_trainer.py` — an offline script that trains a CFR strategy over an abstracted version of the game and serializes it to `cfr_strategy.json`.

2 Hand Evaluation

Rather than hand-writing comparison logic between every pair of hand categories (flush vs. straight, full house vs. flush, etc.), each 5-card hand is scored as a Python tuple

$$\text{score} = (\text{category}, \text{tiebreak}_1, \text{tiebreak}_2, \dots)$$

where $\text{category} \in \{0, \dots, 8\}$ ranges from High Card to Straight Flush. Python's native tuple comparison then handles all comparisons correctly and automatically: the category dominates the comparison, and ties within a category fall through to the kicker values. For example, a full house is encoded as $(6, r_{\text{trips}}, r_{\text{pair}})$, so any full house tuple is guaranteed to exceed any flush tuple $(5, \dots)$ regardless of the specific ranks involved.

Given 7 cards (2 hole cards + 5 community cards), the best possible 5-card hand is found via

$$\text{best_hand}(C) = \max_{S \in \binom{C}{5}} \text{evaluate_5}(S),$$

implemented with `itertools.combinations(cards, 5)` over all 21 combinations, and `collections.Counter` to detect pairs, trips, quads, and full houses from rank frequency counts.

Correctness. The evaluator was verified with a dedicated test suite (`test_hand_evaluator.py`) covering all 9 hand categories, the wheel straight (A-2-3-4-5, where the Ace plays low), kicker-based tie resolution, and exact split-pot detection — all of which pass.

3 Preflop Strategy: The Chen Formula

A Monte Carlo rollout is a poor tool for preflop decisions: with only 2 of 7 relevant cards known, a rollout of the remaining 5 board cards plus the opponent’s 2 hole cards is both slow and has high variance for a fixed simulation budget. Instead, preflop strength is estimated using the **Chen Formula**, a well-known closed-form heuristic used informally by professional players to rank the 169 canonical starting hands. Given the high card h and low card l of a starting hand:

1. **Base points** from the high card: Ace = 10, King = 8, Queen = 7, Jack = 6; otherwise $\text{rank}(h)/2$.
2. **Pair bonus**: if $h = l$ (a pocket pair), points are doubled, floored at a minimum of 5.
3. **Suited bonus**: +2 if h and l share a suit.
4. **Gap penalty**: for non-pairs, a penalty is subtracted based on the rank gap $g = \text{rank}(h) - \text{rank}(l) - 1$: -1 for $g = 1$, -2 for $g = 2$, -4 for $g = 3$, -5 for $g \geq 4$.
5. **Connector bonus**: +1 if $g \leq 1$ and $h < \text{Queen}$ (extra straight potential for low/medium connectors).

This yields a score roughly in $[0, 20]$, which is then linearly mapped onto an approximate heads-up win probability:

$$P(\text{win}) = 0.30 + 0.55 \times \frac{\min(\text{score}, 20)}{20},$$

calibrated so that pocket Aces (score = 20) map to $P \approx 0.85$ and the weakest hands map to the empirically observed heads-up floor of $P \approx 0.30$ (in heads-up play, even the worst starting hand retains meaningful equity against a random opponent hand, unlike in full-ring play). Table 1 shows this ranking is directionally correct against poker intuition.

Hand	Description	Chen Score	Estimated $P(\text{win})$
AA	Pocket Aces	20.0	0.85
KK	Pocket Kings	16.0	0.74
AKs	Ace-King suited	12.0	0.63
AKo	Ace-King offsuit	10.0	0.57
98s	Suited connector	7.5	0.51
22	Lowest pocket pair	5.0	0.44
72o	Worst starting hand	0.0	0.30

Table 1: Chen Formula scores and mapped win probabilities for representative hands.

This approach reduces preflop decision latency from tens of milliseconds (Monte Carlo) to sub-millisecond, which matters when the tournament arena runs 10,000 hands with multiple decision points each.

4 Postflop Strategy: Monte Carlo Rollouts

Once community cards are visible, there is genuine information to exploit, so postflop win probability is estimated directly via Monte Carlo rollout. For $N = 100$ trials, the estimator samples a

random completion of the board and a random opponent hand from the remaining unseen cards, evaluates both hands with `best_hand`, and accumulates:

$$\hat{P}(\text{win}) = \frac{1}{N} \sum_{i=1}^N \begin{cases} 1 & \text{if my hand beats the sampled opponent hand} \\ 0.5 & \text{if tied} \\ 0 & \text{otherwise} \end{cases}$$

This is an unbiased Monte Carlo estimator of the true win probability conditional on the known cards; its standard error scales as $O(1/\sqrt{N})$, which is why $N = 100$ was chosen as a practical trade-off between estimate stability and the $\sim 10,000$ -hand runtime budget.

5 Decision Rule: Pot Odds and Expected Value

Given a win probability estimate $\hat{P}$, the baseline decision rule compares it against the pot odds required to profitably continue:

$$P_{\text{required}} = \frac{\text{amount_to_call}}{\text{pot_size} + \text{amount_to_call}}.$$

If $\hat{P} > P_{\text{required}}$, calling has positive expected value:

$$\mathbb{E}[\text{EV}_{\text{call}}] = \hat{P} \cdot (\text{pot_size} + \text{amount_to_call}) - (1 - \hat{P}) \cdot \text{amount_to_call} > 0.$$

Hands with $\hat{P} > 0.75$ raise for value; hands below P_{required} (with a small 0.05 tolerance buffer to absorb Monte Carlo noise) fold. When `amount_to_call` = 0 (a free check), the agent never folds, since checking has zero downside.

5.1 Controlled Bluffing

A purely EV-based policy is exploitable: an opponent that observes many hands can learn that this agent only continues with genuine equity, and can profitably bet into it with any two cards on later streets. To counter this, whenever the EV heuristic would otherwise fold to a bet that costs less than 20% of the current stack, the agent raises instead with probability 0.08 (and folds otherwise). This bounded, low-frequency bluff limits the maximum damage a bad bluff can do while still making the agent’s behavior non-deterministic and harder to exploit — consistent with the fact that optimal poker strategies are inherently mixed strategies rather than pure threshold rules.

6 Counterfactual Regret Minimization

6.1 Motivation and Abstraction

CFR (Zinkevich et al., 2007) is an iterative self-play algorithm that converges to a Nash equilibrium strategy in two-player zero-sum imperfect-information games. Applied to the exact card space of HULHE, the state space is intractably large (billions of information sets). Real production poker agents such as Libratus and Pluribus address this with *card abstraction*: grouping strategically similar hands into a small number of buckets and solving CFR exactly over the abstracted game. This project adopts the same principle at a scale appropriate for a class project.

Bucketing. Each player’s hand strength at any decision point is mapped to one of 5 buckets via

$$\text{bucket} = \min(\lfloor 5 \cdot \hat{P}(\text{win}) \rfloor, 4) \in \{0, 1, 2, 3, 4\},$$

using the same win-probability estimator described above (Chen Formula preflop, Monte Carlo postflop). Bucket 0 represents the weakest hands and bucket 4 the strongest.

Street-to-street evolution. During offline training, a player’s bucket is not tied to literal cards; instead, it evolves as a bounded random walk between streets,

$$b_{t+1} = \text{clip}(b_t + \delta, 0, 4), \quad \delta = \begin{cases} -1 & \text{w.p. } 0.2 \\ 0 & \text{w.p. } 0.6 \\ +1 & \text{w.p. } 0.2 \end{cases}$$

reflecting the intuition that a hand’s relative strength drifts somewhat as community cards are revealed, without the computational cost of tracking literal card combinations through the training loop.

Showdown resolution. Since exact cards are not tracked during training, the probability that bucket b_A beats bucket b_B at showdown is approximated as a monotonic function of the bucket gap $\Delta = b_A - b_B$ (Table 2), in the spirit of an Elo-style win-probability curve: a larger gap implies a higher win probability, and equal buckets imply a fair (50/50 in expectation) contest.

Δ	−4	−3	−2	−1	0	+1	+2	+3	+4
$P(A \text{ wins})$	0.05	0.15	0.25	0.35	0.50	0.65	0.75	0.85	0.95

Table 2: Approximate showdown win probability as a function of bucket gap.

Betting abstraction. Only the *cards* are abstracted — the betting structure is preserved exactly as specified: fixed bet sizes of 2 chips (preflop/flop) and 4 chips (turn/river), a cap of 4 total bets per street, and the specified heads-up turn order.

6.2 Algorithm: External-Sampling Monte Carlo CFR

Vanilla CFR requires a full traversal of the game tree on every iteration, which is unnecessary here since the abstracted game is small. Instead we use **External-Sampling Monte Carlo CFR** (Lanctot et al., 2009): on each iteration, the traversing player explores *all* of their legal actions (needed to compute counterfactual regret), while the opponent’s actions and chance events (bucket transitions) are each sampled once according to the current strategy / transition distribution. This is far cheaper per iteration than vanilla CFR while retaining the same convergence guarantee in expectation.

At each information set (keyed by (`bucket`, `street`, `bets_count`, `facing_bet`)), the strategy is obtained via **regret matching**:

$$\sigma(a) = \frac{\max(R(a), 0)}{\sum_{a'} \max(R(a'), 0)} \quad (\text{or uniform if all regrets are non-positive}),$$

where $R(a)$ is the cumulative counterfactual regret for action a . After training, the *average* strategy (not the final iteration’s strategy) is extracted and used at inference time, since the time-averaged strategy is the one with equilibrium convergence guarantees in CFR.

Training was run for 300,000 iterations, converging to 100 distinct information sets (the full size of the abstracted infoset space) in approximately 154 seconds on a single CPU core. Table 3 shows a representative slice of the learned strategy.

Bucket	Fold	Call	Raise
0 (weakest)	1.00	0.00	0.00
1	0.00	1.00	0.00
2	0.00	1.00	0.00
3	0.00	1.00	0.00
4 (strongest)	0.00	0.00	1.00

Table 3: Learned river strategy facing a bet, by bucket. The strategy is strongly monotonic in hand strength, as expected: the weakest bucket always folds and the strongest always raises for value.

Interestingly, the learned *preflop* strategy facing the blind occasionally favors calling (“limping”) over raising even for moderate-to-strong buckets. While counter-intuitive at first glance, this is a legitimate emergent property of the abstracted game: with only 5 discrete buckets and a bet cap of 4, immediately raising forecloses future betting rounds’ information value, and the equilibrium strategy in this simplified game can favor delaying aggression. This is a known phenomenon in CFR research — equilibrium strategies are frequently non-obvious and do not always match single-street heuristic intuition, which is part of what makes game-theoretic solutions valuable over hand-coded rules.

6.3 Inference-Time Integration

At decision time, the current game state is mapped to an information-set key and looked up in the trained strategy table. If found, the agent **samples** an action proportionally to the trained probabilities (rather than taking the arg-max), since playing the exact mixed strategy is the theoretically correct way to deploy an average CFR strategy. If the infoset is not present in the table — or the strategy file fails to load for any reason — the agent falls back to the EV heuristic of Section 5, guaranteeing the agent never crashes and always returns a legal action.

A practical limitation. The `get_action()` interface mandated by the assignment does not expose the betting history of the current street (i.e., how many bets/raises have already occurred). Since the bot instance persists across all decisions within a hand and heads-up turns strictly alternate, this is reconstructed internally: if the agent is asked to act again on the same street without the board changing, this can only mean the opponent raised (a call or fold would have ended the street or hand). Combined with tracking the agent’s own chosen actions, this yields an accurate — though not perfectly exact for the very first decision after an opponent’s opening action on a street where the agent acts second — reconstruction of the betting count used for the infoset lookup. This approximation is a deliberate, documented trade-off rather than an oversight.

7 Results

A standalone local game engine (`simulate.py`, not part of the submission) implementing the exact assignment rules was used to evaluate the agent against a random baseline opponent (which calls most actions, folding or raising with low fixed probability). Over 2,000 simulated hands:

- Net result: approximately +0.8 big blinds per hand on average.
- The agent reliably folds weak hands preflop and postflop, raises strong made hands (e.g. always raising a river nut flush), and never folds when facing a free check.

While a random baseline is a weak benchmark, it confirms the basic soundness of the decision pipeline end-to-end (evaluator $\rightarrow$ probability estimate $\rightarrow$ CFR/EV decision) before deployment into the tournament arena against peer-submitted agents.

8 Limitations and Future Work

- **Bucket granularity.** Five buckets is a coarse abstraction; a natural extension is increasing bucket count, or using a continuous equity value with function approximation (e.g. a small neural network) instead of discrete buckets.
- **Independent bucket evolution.** The random-walk bucket transition model treats each player’s hand strength as evolving independently, when in reality both players’ equities are correlated through the shared community cards. A joint transition model, estimated empirically via sampling, would be more faithful.
- **Betting history reconstruction.** As discussed in Section 6.3, the bet-count reconstruction is exact in most cases but can be approximate on a player’s first decision of a street when they act second. Exposing betting history explicitly in the interface (or tracking a hand ID) would remove this limitation entirely.
- **Opponent modeling.** The current agent does not adapt to a specific opponent’s observed tendencies within a match. An extension using online opponent exploitation (e.g. adjusting away from equilibrium once a leak is detected) could improve performance against weak opponents in the tournament, at the cost of exploitability against strong ones.

9 Conclusion

This project implements a complete, from-scratch HULHE game engine and decision agent, layering an exact hand evaluator, a closed-form preflop heuristic (Chen Formula), Monte Carlo postflop equity estimation, and an abstracted CFR-trained strategy with a provably safe EV-heuristic fallback. Each layer is independently unit-tested, and the complete pipeline was validated via a from-scratch local game-engine simulation prior to submission. The CFR component, while trained over a necessarily simplified abstraction, reproduces qualitatively correct poker behavior (monotonic fold/call/raise thresholds by hand strength) and illustrates the core game-theoretic principles — regret minimization and equilibrium mixed strategies — that underlie state-of-the-art poker agents.